Kiosky

Sistema web para la gestion integral de kioscos

Documentacion final del proyecto

Aaron Brumat, Elian Cavenatti, Lucas Beim y Leonardo Vertali

Instituto Superior Juan Jose Urquiza

Carrera de Desarrollo de Software

Practica Profesionalizante 1

Profesor Diego Servan

13 de octubre de 2026

Indice

1. Sintesis 7

2. Tema 8

3. Situacion problematica 8

4. Justificacion 9

5. Antecedentes 9

6. Objetivo general 10

7. Objetivos especificos 10

8. Organizacion del proyecto 11

9. Recursos 11

9.1. Recursos humanos 12

9.2. Recursos de hardware 12

9.3. Recursos de software 12

10. Funciones de los integrantes 13

10.1. Lucas Beim 13

10.2. Elian Cavenatti 13

10.3. Leonardo Vertali 14

10.4. Aaron Brumat 14

11. Entrevista de relevamiento 14

11.1. Preguntas y respuestas 15

12. Conclusion del relevamiento 16

13. Analisis funcional 17

13.1. Actores del sistema 17

13.2. Requerimientos funcionales 17

13.3. Requerimientos no funcionales 18

14. Casos de uso 19

14.1. CU-01. Registrar venta 19

14.2. CU-02. Administrar productos 20

14.3. CU-03. Registrar producto vencido o danado 21

14.4. CU-04. Consultar reportes de ventas 21

14.5. CU-05. Consultar productos mas vendidos 22

15. Historias de usuario 23

16. Wireframes 23

16.1. Pantalla de inicio de sesion 24

16.2. Tablero principal 24

16.3. Gestion de productos 24

16.4. Registrar venta 24

16.5. Reportes 25

17. Entorno de desarrollo asistido por inteligencia artificial 25

17.1. Forma de uso en el proyecto 25

17.2. Modelo de inteligencia artificial utilizado 25

17.3. Limites observados del uso de inteligencia artificial 26

18. Repositorio conectado al proyecto 26

19. Gestion de tickets del proyecto 27

20. Registro de prompts por commit 28

20.1. Aaron Brumat. Ticket KIO-08 28

20.2. Leonardo Vertali. Tickets KIO-03, KIO-04 y KIO-05 29

20.3. Lucas Beim. Tickets KIO-06 y KIO-07 30

20.4. Refuerzo de la seguridad del inicio de sesion. Ticket KIO-01 30

21. Modelo de datos 31

21.1. Diagrama entidad-relacion 31

21.2. Relaciones principales 32

22. Diccionario de datos 32

22.1. Tabla usuarios 32

22.2. Tabla categorias 33

22.3. Tabla productos 33

22.4. Tabla medios_pago 33

22.5. Tabla ventas 34

22.6. Tabla detalle_venta 34

22.7. Tabla movimientos_stock 34

22.8. Tabla bajas_producto 35

23. Reglas de integridad 35

24. Diseno de clases 36

25. Flujo entre capas 37

26. Metodologia de desarrollo 38

26.1. Organizacion del trabajo 38

26.2. Ciclo de trabajo de cada cambio 39

26.3. Practicas adoptadas y su motivo 39

27. Planificacion de tareas 40

28. Diagrama de Gantt 41

29. Gestion de riesgos 41

30. Plan de testing 42

30.1. Alcance 42

30.2. Ambiente de pruebas 43

30.3. Estrategia y tipos de prueba 44

30.4. Juego de datos y valores de referencia 44

30.5. Criterio de aceptacion de la entrega 45

31. Casos de prueba 45

31.1. Casos representativos 46

32. Pruebas automatizadas 47

33. Integracion continua..... 48

34. Registro de defectos y auditoria interna..... 49

34.1. Clasificacion utilizada..... 49

34.2. Defectos registrados..... 49

34.3. Hallazgos de la auditoria interna..... 50

35. Entorno operativo..... 51

35.1. Requisitos de hardware y software..... 51

36. Implementacion y despliegue..... 52

36.1. Tecnologias utilizadas..... 52

36.2. Estructura del sistema entregado..... 53

36.3. Puesta en marcha del sistema..... 53

36.4. Que se puede comprobar en la demostracion..... 54

37. Evidencia de participacion de los integrantes..... 55

38. Conclusiones..... 56

39. Glosario..... 57

40. Referencias..... 59

40.1. Recursos de formacion consultados..... 59

1. Sintesis

El presente proyecto consiste en el analisis, diseno y desarrollo de un sistema web denominado Kiosky, orientado a mejorar la administracion de pequenos comercios del rubro kiosco.

Actualmente, muchos kioscos continuan realizando el control de ventas, stock y mercaderia mediante registros manuales, lo que genera errores, perdida de tiempo y dificultades para conocer el estado real del negocio. Frente a esta situacion se propone desarrollar una aplicacion web que permita automatizar dichos procesos mediante una interfaz sencilla, accesible y facil de utilizar.

El sistema brinda funcionalidades para administrar productos, registrar ventas, controlar el inventario en tiempo real, gestionar diferentes medios de pago, identificar productos proximos a vencer y generar reportes de ventas e inventario. De esta manera se busca optimizar la administracion diaria del comercio, mejorar la toma de decisiones y reducir las perdidas ocasionadas por errores humanos.

El proyecto fue desarrollado aplicando los conocimientos adquiridos durante la carrera de Desarrollo de Software, siguiendo una metodologia de trabajo colaborativa entre los integrantes del equipo y manteniendo una comunicacion permanente con el cliente para validar los requerimientos del sistema.

Al cierre de esta entrega el sistema se encuentra implementado y en funcionamiento: cuenta con ocho pantallas operativas, una interfaz de programacion de aplicaciones de veintiun rutas, una base de datos relacional de ocho tablas, 65 pruebas automatizadas, 129 casos de prueba documentados y un entorno de integracion continua que ejecuta las verificaciones en cada cambio enviado al repositorio.

2. Tema

Desarrollo de un sistema web para la gestion administrativa de un kiosco, enfocado en el control de ventas, administracion de stock, seguimiento de productos y generacion de reportes para mejorar la eficiencia operativa del negocio.

3. Situacion problematica

En numerosos pequenos comercios, especialmente kioscos de barrio, la administracion diaria continua realizandose mediante cuadernos, anotaciones manuales o planillas basicas. Este metodo dificulta mantener un control preciso del inventario y de las ventas realizadas durante la jornada.

El cliente entrevistado manifesto que frecuentemente desconoce el stock disponible de determinados productos, detectando faltantes unicamente cuando un cliente solicita un articulo que ya no se encuentra disponible. Asimismo, el registro manual de ventas demanda tiempo adicional durante la atencion, aumentando la posibilidad de cometer errores en la carga de informacion.

Otro inconveniente identificado corresponde al control de productos proximos a vencer o danados. Actualmente esta tarea se realiza manualmente, lo que incrementa el riesgo de perdidas economicas debido a productos vencidos que permanecen registrados como disponibles para la venta.

Ademas, el comercio no dispone de reportes que permitan conocer cuales son los productos mas vendidos, el rendimiento diario del negocio o la evolucion de las ventas en diferentes periodos, dificultando la toma de decisiones comerciales.

Como consecuencia de estas situaciones, el propietario dedica mas tiempo a tareas administrativas, disminuye la eficiencia del negocio y aumenta la probabilidad de errores operativos.

4. Justificacion

La implementacion de un sistema de gestion permite digitalizar y automatizar los principales procesos administrativos del kiosco, reemplazando los registros manuales por una plataforma centralizada que facilite el acceso a la informacion en tiempo real.

El sistema contribuye a disminuir errores humanos en la administracion del inventario, optimizar el tiempo de atencion al cliente y mejorar el control de la mercaderia disponible. Asimismo, posibilita generar reportes estadisticos sobre ventas y productos mas comercializados, informacion que resulta fundamental para la planificacion de compras y la administracion del negocio.

Desde el punto de vista academico, el desarrollo de este proyecto permitio aplicar conocimientos adquiridos durante la carrera relacionados con analisis de requerimientos, diseno de sistemas, desarrollo web, bases de datos, testing y documentacion tecnica, simulando el ciclo de vida de un proyecto real de software.

5. Antecedentes

Actualmente existen diferentes soluciones comerciales destinadas a la administracion de comercios minoristas, entre las cuales pueden mencionarse SISTAR, Maxirest, Tango Gestion y diversos sistemas de punto de venta utilizados por supermercados y negocios.

Sin embargo, muchas de estas aplicaciones presentan caracteristicas que dificultan su adopcion por parte de pequenos kioscos:

- requieren licencias de pago elevadas;
- incluyen funcionalidades que exceden las necesidades de un comercio pequeno;
- poseen interfaces complejas para usuarios con escasos conocimientos informaticos;
- en algunos casos requieren instalaciones locales y mantenimiento especializado.

A partir de este analisis surge la propuesta de desarrollar Kiosky, un sistema web simple, intuitivo y adaptable a pequenos comercios, que contemple exclusivamente las funcionalidades necesarias para la administracion cotidiana de un kiosco.

6. Objetivo general

Desarrollar un sistema web que permita gestionar de forma eficiente las ventas, el stock y la administracion general de un kiosco, optimizando los procesos operativos y mejorando el acceso a la informacion del negocio.

7. Objetivos especificos

- Automatizar el registro de ventas realizadas diariamente.
- Administrar el inventario en tiempo real.
- Permitir el alta, modificacion y baja de productos.
- Gestionar distintos medios de pago utilizados por el comercio.
- Registrar productos vencidos o danados para actualizar automaticamente el stock.
- Generar reportes de ventas por periodos determinados.
- Identificar los productos con mayor demanda para facilitar la reposicion de mercaderia.

- Desarrollar una interfaz intuitiva que pueda ser utilizada sin conocimientos tecnicos.

8. Organizacion del proyecto

Para el desarrollo del sistema se trabajo siguiendo una metodologia de desarrollo iterativa, que permitio obtener avances parciales revisados periodicamente por el cliente.

El proyecto se organizo en las siguientes etapas:

1. **Relevamiento de informacion.** Se identificaron las necesidades del cliente mediante entrevistas y analisis de los procesos actuales del negocio.
2. **Analisis de requerimientos.** Se documentaron las funcionalidades que debe cumplir el sistema, definiendo actores, requerimientos funcionales y no funcionales, casos de uso e historias de usuario.
3. **Diseno.** Se elaboraron los wireframes de las pantallas principales y se diseno la estructura de la base de datos y la arquitectura general del sistema.
4. **Desarrollo.** Se implementaron los modulos correspondientes al sistema utilizando tecnologias web modernas.
5. **Pruebas.** Se realizaron pruebas funcionales, manuales y automatizadas, para verificar que cada requerimiento se cumpla correctamente y corregir los defectos detectados antes de la entrega.
6. **Entrega.** Finalmente se presenta el sistema al cliente en funcionamiento, acompanado de su documentacion tecnica y funcional.

9. Recursos

Para el desarrollo del sistema Kiosky fue necesario contar con recursos humanos, tecnologicos y fisicos que permitieran llevar adelante cada una de las etapas del proyecto, desde el relevamiento de requerimientos hasta las pruebas finales del sistema.

9.1. Recursos humanos

El equipo esta integrado por cuatro desarrolladores, quienes asumieron responsabilidades especificas segun las necesidades del proyecto.

Tabla 1
Integrantes del equipo y rol principal

Integrante	Rol principal
Aaron Brumat	Testing, documentacion y aseguramiento de la calidad
Elian Cavenatti	Integracion de funcionalidades
Leonardo Vertali	Desarrollo backend y base de datos
Lucas Beim	Desarrollo frontend y diseno visual

Nota. Ademas del equipo de desarrollo participo el cliente, encargado de validar los requerimientos y aprobar las distintas entregas del proyecto.

9.2. Recursos de hardware

- Computadoras personales o notebooks con capacidad suficiente para ejecutar herramientas de desarrollo.
- Conexion estable a Internet.
- Dispositivos moviles para realizar pruebas de visualizacion adaptable.
- Perifericos basicos: teclado, mouse y monitor.

9.3. Recursos de software

Las herramientas seleccionadas para el desarrollo del proyecto son:

- Visual Studio Code como entorno de desarrollo.

- React para el desarrollo del frontend, junto con HTML5, CSS3 y JavaScript.
- Node.js como entorno de ejecucion y Express como marco de trabajo para la interfaz de programacion de aplicaciones.
- MySQL como gestor de base de datos.
- Git y GitHub para el control de versiones, la revision de codigo y la integracion continua.
- Google Chrome y Mozilla Firefox para las pruebas del sistema.
- Figma para el diseno de los wireframes y Lucidchart para el diagrama entidad-relacion y el diagrama de Gantt.
- Trello para la gestion de tickets.
- Herramientas de inteligencia artificial como asistencia al desarrollo.

La eleccion de estas tecnologias responde a su amplia utilizacion en el desarrollo web moderno, su disponibilidad gratuita y la experiencia previa del equipo con dichas herramientas.

10. Funciones de los integrantes

10.1. Lucas Beim

- Desarrollo de la interfaz grafica.
- Diseno de la experiencia de usuario.
- Implementacion de componentes visuales.
- Tratamiento de la expiracion de sesion en pantalla y mejoras de accesibilidad por teclado.

10.2. Elian Cavenatti

- Desarrollo de funcionalidades tanto del frontend como del backend.

- Integracion entre la interfaz y la base de datos.
- Correccion de incidencias detectadas durante el desarrollo.
- Refuerzo de la seguridad del inicio de sesion y correccion del defecto de desfasaje horario entre la aplicacion y la base de datos.

10.3. Leonardo Vertali

- Diseno de la base de datos.
- Elaboracion del modelo de datos y del diagrama entidad-relacion.
- Implementacion del esquema, el procedimiento almacenado de bajas y las vistas analiticas.
- Unificacion del esquema en un unico script de creacion.

10.4. Aaron Brumat

- Elaboracion del plan de pruebas funcionales y de los casos de prueba por requerimiento.
- Ejecucion de las pruebas manuales y registro de los defectos detectados.
- Automatizacion de los casos criticos del plan de pruebas.
- Configuracion de la integracion continua.
- Redaccion de la documentacion tecnica y funcional del proyecto.
- Preparacion del entorno de demostracion del sistema.

11. Entrevista de relevamiento

El objetivo de la entrevista fue conocer el funcionamiento actual del kiosco, identificar las principales dificultades que presenta el propietario durante la administracion del negocio y recopilar los requerimientos necesarios para el desarrollo del sistema.

Tabla 2
Datos de la entrevista de relevamiento

Dato	Detalle
Lugar	Local comercial del cliente
Entrevistador	Equipo de desarrollo
Entrevistado	Propietario del kiosco
Modalidad	Entrevista semiestructurada de ocho preguntas abiertas

11.1. Preguntas y respuestas

Pregunta 1. Como administra actualmente las ventas del negocio?

Las ventas se registran principalmente de forma manual utilizando un cuaderno y, en algunas ocasiones, una planilla electronica. Este procedimiento requiere tiempo y suele generar errores al momento de realizar los controles diarios.

Pregunta 2. Como controla el stock de los productos?

El control se realiza revisando fisicamente la mercaderia disponible. En varias oportunidades el faltante de productos se detecta recien cuando un cliente intenta comprarlos.

Pregunta 3. Que problemas encuentra con este metodo de trabajo?

Existe una perdida considerable de tiempo al registrar las ventas, dificultades para conocer el stock real y una mayor probabilidad de cometer errores en la administracion del negocio.

Pregunta 4. Que funcionalidades considera indispensables para un sistema de gestion?

El propietario considera fundamental poder registrar ventas rapidamente, controlar automaticamente el stock, administrar productos y consultar reportes de ventas.

Pregunta 5. Necesita controlar productos proximos a vencer?

Si. Actualmente ese control se realiza manualmente, lo que ocasiona perdidas economicas cuando algun producto vence sin ser detectado a tiempo.

Pregunta 6. Que informacion le gustaria consultar diariamente?

Monto vendido durante el dia, productos con bajo stock, articulos mas vendidos y productos proximos a vencer.

Pregunta 7. Considera importante obtener reportes automaticos?

Si. Los reportes facilitarian el control del negocio y permitirian tomar mejores decisiones al momento de realizar nuevas compras.

Pregunta 8. Que espera obtener con la implementacion del sistema?

Una administracion mas organizada, mayor rapidez durante las ventas, reduccion de errores y un mejor control general del comercio.

12. Conclusion del relevamiento

La informacion obtenida durante la entrevista permitio identificar que el principal problema del cliente radica en la utilizacion de procesos manuales para administrar el kiosco.

Se detecto la necesidad de contar con una herramienta que automatice el registro de ventas, actualice el inventario en tiempo real, permita gestionar diferentes medios de pago y facilite la consulta de informacion relevante para la administracion del negocio.

Asimismo, el cliente manifesto interes en disponer de reportes estadisticos y alertas relacionadas con el stock y los productos proximos a vencer, funcionalidades que fueron contempladas dentro del alcance del sistema Kiosky.

Cabe senalar que las ocho necesidades expresadas por el cliente se tradujeron en los catorce requerimientos funcionales que se detallan en la seccion siguiente, y que cada uno de ellos cuenta con casos de prueba asociados en el plan de testing. Esa cadena, de la necesidad al requerimiento y del requerimiento al caso de prueba, es la que permite afirmar que el sistema entregado responde a lo relevado.

13. Analisis funcional

13.1. Actores del sistema

Durante el analisis del proyecto se identificaron los siguientes actores que interactuan con el sistema.

Tabla 3
Actores identificados

Actor	Descripcion
Propietario	Administra completamente el sistema: gestiona productos, ventas, reportes y configuracion.
Empleado	Registra ventas y consulta la informacion autorizada por el propietario.
Sistema	Ejecuta procesos automaticos como la actualizacion de stock, la generacion de reportes y el control de productos vencidos.

13.2. Requerimientos funcionales

RF-01. El sistema debera permitir iniciar sesion mediante usuario y contrasena.

RF-02. El sistema debera permitir registrar ventas de uno o varios productos.

RF-03. El sistema debera actualizar automaticamente el stock despues de cada venta.

RF-04. El sistema debera permitir registrar productos nuevos.

RF-05. El sistema debera permitir modificar los datos de un producto existente.

RF-06. El sistema debera permitir eliminar productos del inventario.

RF-07. El sistema debera permitir buscar productos por nombre o codigo de barras.

RF-08. El sistema debera permitir registrar distintos medios de pago.

RF-09. El sistema debera generar reportes de ventas por dia, semana y mes.

RF-10. El sistema debera mostrar los productos mas vendidos.

RF-11. El sistema debera registrar productos danados o vencidos.

RF-12. El sistema debera descontar automaticamente dichos productos del inventario.

RF-13. El sistema debera mostrar alertas cuando el stock sea bajo.

RF-14. El sistema debera permitir consultar el historial de ventas.

13.3. Requerimientos no funcionales

RNF-01. La interfaz debera ser intuitiva y facil de utilizar.

RNF-02. El sistema debera responder a las acciones del usuario en un tiempo inferior a tres segundos en condiciones normales.

RNF-03. La aplicacion debera ser compatible con los navegadores mas utilizados: Google Chrome, Microsoft Edge y Mozilla Firefox.

RNF-04. El sistema debera proteger el acceso mediante autenticacion de usuarios.

RNF-05. La informacion almacenada debera mantenerse integra y consistente dentro de la base de datos.

RNF-06. El sistema debera ser desarrollado siguiendo una arquitectura que facilite futuras ampliaciones y tareas de mantenimiento.

14. Casos de uso

14.1. CU-01. Registrar venta

Actor principal: propietario o empleado.

Objetivo: permitir registrar una venta realizada en el kiosco, actualizar automaticamente el stock y almacenar la informacion para futuras consultas.

Precondiciones:

- El usuario debe haber iniciado sesion.
- Los productos deben estar registrados en el sistema.
- Debe existir stock suficiente de los productos seleccionados.

Flujo principal:

7. El usuario selecciona la opcion Nueva Venta.
8. El sistema muestra el formulario de venta.
9. El usuario busca uno o mas productos mediante nombre o codigo de barras.
10. El sistema muestra los productos encontrados.
11. El usuario selecciona las cantidades correspondientes.
12. El sistema calcula automaticamente el importe total.
13. El usuario selecciona el medio de pago.
14. El sistema registra la venta.
15. El sistema actualiza automaticamente el stock.
16. El sistema confirma la operacion.

Flujos alternativos:

- **FA-1. Stock insuficiente.** El sistema informa que no existe stock suficiente y la venta no puede finalizar hasta modificar la cantidad solicitada.

- **FA-2. Producto inexistente.** El sistema informa que el producto no se encuentra registrado.

Postcondiciones: la venta queda almacenada, el stock queda actualizado y la operacion queda registrada en el historial.

14.2. CU-02. Administrar productos

Actor principal: propietario.

Objetivo: administrar el inventario del kiosco permitiendo registrar nuevos productos y modificar la informacion existente.

Precondiciones: el usuario debe haber iniciado sesion.

Flujo principal:

17. El usuario ingresa al modulo Productos.
18. El sistema muestra el listado de productos.
19. El usuario selecciona la opcion Agregar Producto.
20. El sistema solicita los datos del nuevo producto.
21. El usuario completa la informacion requerida.
22. El sistema valida los datos.
23. El sistema almacena el producto.
24. El sistema informa que el producto fue registrado correctamente.

Flujos alternativos:

- **FA-1. Modificar producto.** El usuario selecciona un producto existente, el sistema muestra la informacion almacenada, el usuario modifica los datos y el sistema guarda los cambios.

- **FA-2. Eliminar producto.** El usuario selecciona un producto, el sistema solicita confirmacion y, al confirmarse, el producto deja de estar disponible en el inventario. La eliminacion es logica y no fisica, por los motivos que se explican en la seccion 22.

Postcondiciones: el inventario queda actualizado.

14.3. CU-03. Registrar producto vencido o danado

Actor principal: propietario.

Objetivo: registrar productos que ya no pueden comercializarse para mantener actualizado el inventario.

Precondiciones: el usuario inicio sesion y el producto existe en la base de datos.

Flujo principal:

25. El usuario ingresa al modulo Productos Vencidos.
26. El sistema solicita seleccionar un producto.
27. El usuario indica el motivo de la baja.
28. El usuario ingresa la cantidad afectada.
29. El sistema descuenta automaticamente las unidades del stock.
30. El sistema registra la operacion.
31. El sistema muestra un mensaje de confirmacion.

Flujo alternativo FA-1. Cantidad invalida. Si la cantidad ingresada supera el stock disponible, el sistema informa el error y solicita una nueva cantidad.

Postcondiciones: el stock refleja la cantidad real disponible y queda registrada la trazabilidad de la perdida.

14.4. CU-04. Consultar reportes de ventas

Actor principal: propietario.

Objetivo: consultar las ventas realizadas durante un periodo determinado.

Precondiciones: el usuario debe haber iniciado sesion.

Flujo principal:

32. El usuario accede al modulo Reportes.
33. Selecciona el periodo a consultar.
34. El sistema recupera la informacion.
35. El sistema muestra el total vendido, la cantidad de ventas, los medios de pago utilizados y los productos vendidos.

Flujo alternativo. Si no existen ventas durante el periodo seleccionado, el sistema informa que no hay datos disponibles.

Postcondiciones: el usuario obtiene un reporte actualizado del negocio.

14.5. CU-05. Consultar productos mas vendidos

Actor principal: propietario.

Objetivo: obtener estadisticas sobre los productos con mayor demanda.

Precondiciones: deben existir ventas registradas.

Flujo principal:

36. El usuario ingresa al modulo de estadisticas.
37. Selecciona el rango de fechas.
38. El sistema procesa la informacion.
39. El sistema muestra un ranking de productos ordenados por cantidad de unidades vendidas.

Flujo alternativo. Si no existen ventas registradas para el periodo seleccionado, el sistema informa que no puede generar estadisticas.

Postcondiciones: el usuario obtiene informacion util para planificar futuras compras.

15. Historias de usuario

Tabla 4
Historias de usuario y criterios de aceptacion

ID	Titulo	Historia	Criterios de aceptacion
HU-01	Registrar venta	Como propietario del kiosco quiero registrar ventas de forma rapida para reducir el tiempo de atencion al cliente.	Se debe poder buscar productos. El total debe calcularse automaticamente. El stock debe actualizarse luego de cada venta.
HU-02	Administrar productos	Como propietario quiero administrar los productos del inventario para mantener actualizada la informacion del negocio.	Se podran agregar, modificar y dar de baja productos.
HU-03	Controlar el stock	Como propietario quiero conocer el stock disponible en tiempo real para evitar faltantes de mercaderia.	El stock debe actualizarse automaticamente. Deben mostrarse alertas cuando el stock sea bajo.
HU-04	Consultar reportes	Como propietario quiero visualizar reportes de ventas para analizar el rendimiento del negocio.	Se podran consultar ventas por dia, semana y mes. Los reportes deberan mostrar el total vendido.
HU-05	Productos mas vendidos	Como propietario quiero conocer cuales son los productos mas vendidos para planificar las compras futuras.	El sistema debera mostrar un ranking de productos. La consulta debera poder filtrarse por fechas.
HU-06	Productos vencidos	Como propietario quiero registrar productos vencidos o danados para mantener actualizado el inventario.	El sistema descontara automaticamente el stock. La operacion quedara registrada.

16. Wireframes

Para el proyecto Kiosky se disenaron los wireframes de las cinco pantallas principales utilizando la herramienta Figma. Las pantallas implementadas respetan la estructura propuesta en el diseno.

16.1. Pantalla de inicio de sesion

Permite el acceso al sistema mediante usuario y contrasena, garantizando que solo personas autorizadas puedan utilizar las funcionalidades del sistema. Sus elementos principales son el logo del sistema, el campo de usuario, el campo de contrasena, el boton de ingreso y los mensajes de validacion.

16.2. Tablero principal

Es la pantalla inicial luego del inicio de sesion y muestra informacion resumida del negocio. Sus elementos principales son el menu lateral, las ventas del dia, los productos con stock bajo, los productos proximos a vencer, los accesos rapidos y los indicadores generales.

16.3. Gestion de productos

Permite administrar el inventario del kiosco. Sus elementos principales son el buscador, la tabla de productos, el boton de agregar producto, los botones de editar y dar de baja, y el stock disponible de cada articulo.

16.4. Registrar venta

Pantalla destinada a realizar ventas. Sus elementos principales son el buscador por nombre, el buscador por codigo de barras, la lista de productos seleccionados, la cantidad, el total, el medio de pago y el boton de confirmacion.

16.5. Reportes

Permite consultar informacion estadistica. Sus elementos principales son la seleccion de fechas, el total vendido, los productos mas vendidos y el desglose por medios de pago.

17. Entorno de desarrollo asistido por inteligencia artificial

Herramienta seleccionada: Visual Studio Code con asistentes de inteligencia artificial integrados. Visual Studio Code es el entorno de desarrollo utilizado por el equipo para implementar el sistema web Kiosky. Los asistentes de inteligencia artificial se incorporan dentro del entorno para sugerencias de codigo, consultas tecnicas, explicacion de errores, refactorizacion y apoyo en la documentacion.

17.1. Forma de uso en el proyecto

- Generar borradores de componentes React para pantallas como inicio de sesion, tablero, gestion de productos, registro de ventas y reportes.
- Consultar alternativas para estructurar funciones de backend en Node.js y consultas SQL para MySQL.
- Pedir explicaciones de errores del entorno, mensajes de consola y fallas de integracion.
- Solicitar casos de prueba y criterios de aceptacion para validar los requerimientos funcionales.
- Revisar nombres de variables, modularizacion y legibilidad antes de confirmar los cambios.
- Registrar en cada commit el prompt usado, el ticket asociado y el cambio realizado.

17.2. Modelo de inteligencia artificial utilizado

Tabla 5
Caracterizacion del modelo de inteligencia artificial empleado

Aspecto	Descripcion
Tipo de modelo	Modelo de lenguaje grande capaz de interpretar instrucciones en lenguaje natural, generar codigo, explicar errores, resumir documentacion y asistir en tareas de analisis y testing.
Caracteristicas principales	Buen rendimiento en generacion y revision de codigo, razonamiento paso a paso, trabajo con el contexto del proyecto, redaccion tecnica y soporte para depuracion. Las respuestas deben ser revisadas por el equipo antes de incorporarse al repositorio.
Costo y disponibilidad	Las herramientas utilizadas ofrecen modalidad gratuita con limites y planes pagos con mayores capacidades. El equipo utiliza, en particular, el plan para estudiantes que otorga acceso durante un ano.
Tiempo de respuesta	Valores observados durante el proyecto: entre 2 y 10 segundos para consultas simples, y entre 10 y 60 segundos o mas para tareas de codigo, explicacion o refactorizacion, segun la longitud del prompt, el plan utilizado, la conexion y la carga del servicio.
Uso responsable	La inteligencia artificial se utiliza como asistencia. El equipo valida el codigo, ejecuta las pruebas, ajusta la solucion al alcance del proyecto y deja trazabilidad del prompt en los commits y en los documentos.

17.3. Limites observados del uso de inteligencia artificial

El trabajo con asistentes de inteligencia artificial dejo una leccion concreta que conviene documentar, porque es la que da sentido al riesgo R-05 de la seccion 29. Dos de los tres defectos registrados en este proyecto, DEF-002 y DEF-003, corresponden a codigo generado con asistencia que superaba las verificaciones a simple vista y que resultaba incorrecto en situaciones de borde: una comparacion de fechas que interpretaba el vencimiento en un huso horario distinto al local, y un desfasaje entre el reloj de la base de datos y el de la aplicacion.

Ninguno de los dos defectos fue detectado leyendo el codigo. Ambos aparecieron al escribir pruebas automatizadas que ejercitaban el caso limite. Esa es la razon por la cual el equipo adopto la practica de acompanar cada funcionalidad generada con asistencia con al menos una prueba que verifique su caso de borde, y no unicamente su camino feliz.

18. Repositorio conectado al proyecto

Repositorio seleccionado: GitHub.

Tabla 6
Datos del repositorio del proyecto

Dato	Valor
Direccion publica	https://github.com/47103032-web/Proyecto-Kiosky
Cuentas de los integrantes	43126754, 45505303, 47103032 y 45265475, todas del dominio terciariourquiza.edu.ar
Rama principal	main
Estrategia de ramas	Cada ticket de Trello da origen a una rama independiente que representa el objetivo general de la tarea. Sobre su rama asignada, cada integrante sube al menos tres commits que funcionan como avance progresivo hacia el objetivo, y la integracion a la rama principal se realiza mediante solicitud de incorporacion de cambios.
Control de calidad	Toda solicitud de incorporacion ejecuta automaticamente las pruebas del proyecto antes de poder integrarse. El detalle se desarrolla en la seccion 33.

Fundamentacion de la eleccion:

- Permite alojar el codigo fuente, registrar commits con autor, fecha y mensaje, y conservar un historial verificable.
- Facilita la colaboracion mediante ramas, solicitudes de incorporacion de cambios, incidencias, revisiones y permisos por colaborador.
- Se integra con Visual Studio Code, con los asistentes de inteligencia artificial y con Trello mediante enlaces en los commits y en las tarjetas.
- Incluye un servicio de integracion continua sin costo adicional, que el proyecto utiliza para ejecutar las pruebas en cada cambio.

19. Gestion de tickets del proyecto

Gestor seleccionado: Trello. La eleccion responde a que permite organizar tareas en tableros Kanban simples, adecuados para un equipo academico pequeno; cada ticket puede incluir responsable, descripcion, lista de verificacion, fecha, etiquetas, adjuntos y comentarios; y la

version gratuita incluye tarjetas ilimitadas, fechas de vencimiento, responsables y tableros suficientes para este proyecto.

Tabla 7
Tickets del proyecto y asignacion

Ticket	Objetivo	Responsable	Caso de uso
KIO-01	Autenticacion y seguridad de la sesion	Elian Cavenatti	RF-01
KIO-02	Registro de ventas e integracion con la base de datos	Elian Cavenatti	CU-01
KIO-03	Esquema de la base de datos	Leonardo Vertali	CU-02
KIO-04	Productos vencidos o danados	Leonardo Vertali	CU-03
KIO-05	Reportes de ventas y productos mas vendidos	Leonardo Vertali	CU-04 y CU-05
KIO-06	Interfaz de usuario y pantallas principales	Lucas Beim	CU-01 a CU-05
KIO-07	Tablero, alertas y experiencia de uso	Lucas Beim	CU-05
KIO-08	Testing, documentacion y aseguramiento de la calidad	Aaron Brumat	Todos

20. Registro de prompts por commit

Esta seccion da cumplimiento al requisito de trazabilidad entre el prompt utilizado y el cambio incorporado al repositorio. Se consigna, para cada commit, el identificador abreviado que figura en el historial de GitHub, el ticket asociado, el prompt empleado y el resultado obtenido.

El repositorio parte de un commit de base, identificado como 9c5ee99, que incorpora la estructura inicial del sistema correspondiente a los tickets KIO-01 a KIO-07 y que fue realizado por Aaron Brumat durante la integracion inicial del proyecto. Los commits que se detallan a continuacion corresponden al trabajo posterior de cada integrante sobre su propia rama.

20.1. Aaron Brumat. Ticket KIO-08

Tabla 8
Registro de prompts de Aaron Brumat

Commit	Fecha	Prompt utilizado	Resultado obtenido
1fa5ac0	01/09/2026	Genera un plan de pruebas funcionales para los casos de uso de Kiosky: registrar venta, administrar productos, productos vencidos, reportes y productos mas vendidos. Incluir criterios de aceptacion y casos positivos y negativos.	Plan de pruebas funcionales con 48 casos derivados de los cinco casos de uso, con criterios de aceptacion y casos negativos por flujo alternativo.
359ff74	01/09/2026	Revisa esta lista de requerimientos funcionales de Kiosky y propone casos de prueba manuales para validar cada uno, con datos de entrada y resultado esperado.	81 casos de prueba manuales, organizados por requerimiento funcional y con matriz de trazabilidad hacia los RF.
4ee9b17	01/09/2026	Ayudame a redactar la documentacion tecnica de testing para Kiosky, incluyendo alcance, datos de prueba, resultados esperados, procedimiento de ejecucion y registro de errores.	Documentacion tecnica de testing con alcance, ambiente, juego de datos, valores de referencia, criterio de aceptacion de la entrega y plantilla de registro de defectos.
98067ad	10/09/2026	Automatiza con el ejecutor de pruebas nativo de Node los casos criticos del plan de pruebas de Kiosky, sin agregar dependencias externas.	65 pruebas automatizadas sobre dominio, ventas, stock y reportes. Durante su desarrollo se detecto el defecto DEF-002.
4af940c	20/09/2026	Configura la integracion continua del repositorio para que ejecute las pruebas del backend, compile el frontend y verifique que el esquema de la base de datos se aplica sobre una base vacia.	Flujo de integracion continua con tres trabajos, descrito en la seccion 33.
b362065	20/09/2026	Registra en la documentacion tecnica de testing el defecto de desfasaje horario entre el reloj de la base de datos y el de la aplicacion, con pasos para reproducirlo.	Ficha completa del defecto DEF-003, de severidad alta, con pasos de reproduccion y verificacion posterior.
2b98b36	12/10/2026	Prepara el entorno de demostracion: que el sistema pueda ejecutarse sin un servidor de base de datos, siempre desde el mismo juego de datos, y que la interfaz avise en pantalla cuando corre en ese modo para que la condicion no quede oculta.	Entorno de demostracion descrito en la seccion 36, con el aviso de modo en la interfaz.
9c2f048	13/10/2026	Arma el documento de entrega final con las secciones que pide la consigna, respetando normas APA: caratula, indice automatico, tablas con titulo y nota, y referencias con sangria francesa.	Documento de entrega final, en formato editable y en PDF.
b3b5f7e	15/10/2026	Las ventas se guardan con NOW() del servidor de MySQL y los periodos de los reportes se calculan con la fecha local del proceso. Hay que alinear la zona horaria de cada conexion del pool usando el evento de conexion, y no una sola vez al arrancar.	Cierre del defecto DEF-003, de severidad alta, con la verificacion que detalla la seccion 34.

20.2. Leonardo Vertali. Tickets KIO-03, KIO-04 y KIO-05

Tabla 9
Registro de prompts de Leonardo Vertali

Commit	Fecha	Prompt utilizado	Resultado obtenido
0115a6a	10/09/2026	Propone un esquema MySQL para productos, ventas, detalle de ventas, medios de pago y movimientos de stock, con claves foraneas y restricciones de integridad.	Script de creacion de la base de datos con las ocho tablas del modelo, claves foraneas y restricciones de verificacion.
f0154ad	10/09/2026	Crea consultas SQL para registrar productos vencidos o danados, descontar stock y guardar el motivo de la baja en una sola transaccion.	Procedimiento almacenado de bajas de producto, con transaccion, bloqueo de la fila del producto y registro en la tabla de trazabilidad.
b5a3144	17/09/2026	Optimiza las consultas SQL para obtener reportes de ventas y ranking de productos mas vendidos filtrando por periodo.	Dos vistas analiticas y tres indices de apoyo para los reportes.
a73f4ea	17/09/2026	Revisa que el contenido de las vistas y los indices quede efectivamente guardado en el script del repositorio.	Correccion del contenido de las vistas y los indices en el repositorio.

20.3. Lucas Beim. Tickets KIO-06 y KIO-07

Tabla 10
Registro de prompts de Lucas Beim

Commit	Fecha	Prompt utilizado	Resultado obtenido
9074b49	24/09/2026	Crea un componente React para el tablero de Kiosky con ventas del dia, productos con stock bajo y productos proximos a vencer.	Pantalla inicial con los tres paneles de informacion resumida del negocio.
b2a870c	26/09/2026	Propone la estructura de una pantalla React para gestion de productos con buscador, tabla y botones de accion.	Ajustes sobre la pantalla del tablero y su estructura de componentes.
91cb911	11/10/2026	Genera una vista frontend para mostrar productos mas vendidos con filtro por fecha y ranking ordenado por unidades.	Depuracion de la pantalla una vez integrada la version definitiva del tablero.

20.4. Refuerzo de la seguridad del inicio de sesion. Ticket KIO-01

Esta serie de commits corresponde al hallazgo de seguridad descrito en la seccion 34 y fue incorporada por Lucas Beim sobre la rama del ticket KIO-01.

Tabla 11
Registro de prompts del refuerzo de seguridad

Commit	Fecha	Prompt utilizado	Resultado obtenido
65f7ad4	15/10/2026	No hay limite de intentos de inicio de sesion: se pueden probar contrasenas de forma indefinida. Quiero un middleware con una ventana deslizante de cinco intentos fallidos por cuenta cada quince minutos, que responda con el codigo 429 e indique en cuantos minutos se puede volver a intentar, y engancharlo en la ruta de ingreso.	Middleware de limite de intentos de ingreso, enganchado en la ruta de inicio de sesion, con el ajuste de configuracion de entorno que requiere.
3ab41cd	15/10/2026	Revisa la configuracion de la conexion a la base de datos: quedo un valor mal despues de integrar el middleware.	Correccion de la configuracion de la conexion.
c68cd84	15/10/2026	Queda un error en la configuracion de la conexion a la base de datos despues del cambio anterior. Corregilo.	Ajuste final de la configuracion de la conexion.

21. Modelo de datos

El modelo de datos se compone de ocho entidades. Su diseno responde a dos decisiones que atraviesan todo el sistema: la informacion historica no se modifica ni se elimina, y todo movimiento de inventario queda registrado con su motivo.

Tabla 12
Entidades del modelo de datos y su proposito

Entidad	Proposito
usuarios	Registra propietarios y empleados autorizados para usar el sistema.
categorias	Agrupa productos por rubro o tipo de mercaderia.
productos	Almacena datos comerciales y de inventario de cada articulo.
medios_pago	Define formas de cobro: efectivo, tarjeta, transferencia u otras.
ventas	Registra la cabecera de cada operacion de venta.
detalle_venta	Registra los productos, cantidades y subtotales de cada venta.
movimientos_stock	Mantiene la trazabilidad de entradas, salidas y ajustes de stock.
bajas_producto	Registra productos vencidos o danados descontados del inventario.

21.1. Diagrama entidad-relacion

El diagrama entidad-relacion fue elaborado con la herramienta Lucidchart y se adjunta como anexo grafico. Su contenido se expresa de forma equivalente en la tabla de relaciones principales que sigue, donde cada fila indica la cardinalidad y el motivo funcional de la relacion.

21.2. Relaciones principales

Tabla 13
Relaciones entre entidades, cardinalidad y justificacion

Relacion	Cardinalidad	Justificacion funcional
usuarios a ventas	1:N	Un usuario puede registrar muchas ventas y cada venta tiene un responsable identificado.
medios_pago a ventas	1:N	Un medio de pago puede utilizarse en muchas ventas.
ventas a detalle_venta	1:N	Una venta puede incluir uno o varios productos.
productos a detalle_venta	1:N	Un producto puede aparecer en multiples ventas.
categorias a productos	1:N	Una categoria agrupa muchos productos.
productos a movimientos_stock	1:N	Cada producto puede tener varios movimientos de inventario.
productos a bajas_producto	1:N	Cada producto puede tener bajas por vencimiento o dano.

Nota. La clave foranea de detalle_venta hacia productos es la que impide eliminar fisicamente un producto que ya participo de una venta. Esa restriccion es la que obliga a la baja logica descrita en la seccion 22.

22. Diccionario de datos

22.1. Tabla usuarios

Campo	Tipo	Clave	Descripcion
id_usuario	INT	PK	Identificador unico del usuario.
nombre	VARCHAR(100)		Nombre y apellido del propietario o empleado.
email	VARCHAR(120)	UNIQUE	Correo de acceso al sistema.
password_hash	VARCHAR(255)		Contrasena almacenada de forma segura, nunca en texto plano.
rol	ENUM		Perfil del usuario: propietario o empleado.
activo	BOOLEAN		Permite habilitar o deshabilitar usuarios.

22.2. Tabla categorias

Campo	Tipo	Clave	Descripcion
id_categoria	INT	PK	Identificador unico de categoria.
nombre	VARCHAR(80)	UNIQUE	Nombre de la categoria.
descripcion	VARCHAR(255)		Detalle opcional del rubro.
activo	BOOLEAN		Indica si la categoria esta disponible.

22.3. Tabla productos

Campo	Tipo	Clave	Descripcion
id_producto	INT	PK	Identificador unico del producto.
codigo_barra	VARCHAR(50)	UNIQUE	Codigo usado para busqueda rapida. Admite nulo cuando el producto no tiene codigo, por ejemplo en la venta por unidad.
nombre	VARCHAR(120)		Nombre comercial del producto.
descripcion	VARCHAR(255)		Descripcion opcional.
precio	DECIMAL(10,2)		Precio actual de venta.
stock_actual	INT		Cantidad disponible.
stock_minimo	INT		Nivel que dispara la alerta de bajo stock. Su valor por defecto es 0, el mismo que aplica el backend cuando el campo no se informa.
fecha_vencimiento	DATE		Fecha utilizada para el control de vencimientos.
id_categoria	INT	FK	Referencia a categorias. Admite nulo, porque el sistema permite cargar mercaderia sin clasificarla todavia.
activo	BOOLEAN		Implementa la baja logica del producto (RF-06).

Nota. El campo activo merece una aclaracion, porque no figuraba en la primera version de este diccionario y fue incorporado durante la implementacion. Un producto no puede eliminarse fisicamente de la base: la tabla detalle_venta lo referencia por clave foranea, y borrarlo romperia el historial de ventas y la anulacion de ventas previstas en RF-14. La baja, entonces, es logica: al desactivarse, el producto deja de aparecer en el catalogo, en las alertas de stock y en la pantalla de venta, pero las ventas ya registradas se conservan intactas. Esta decision fue documentada a partir del hallazgo descrito en la seccion 34.

22.4. Tabla medios_pago

Campo	Tipo	Clave	Descripcion
id_medio_pago	INT	PK	Identificador unico del medio de pago.

Campo	Tipo	Clave	Descripcion
nombre	VARCHAR(50)	UNIQUE	Efectivo, debito, credito, transferencia u otro.
activo	BOOLEAN		Indica si puede seleccionarse en una venta.

22.5. Tabla ventas

Campo	Tipo	Clave	Descripcion
id_venta	INT	PK	Identificador unico de la venta.
fecha_hora	DATETIME		Momento de registro de la venta.
total	DECIMAL(10,2)		Importe total calculado.
id_usuario	INT	FK	Usuario que registra la venta.
id_medio_pago	INT	FK	Medio de pago seleccionado.
estado	ENUM		Estado de la venta: registrada o anulada.

Nota. La anulacion de una venta no borra el registro: cambia su estado a anulada y repone el stock. De ese modo el historial conserva la operacion y su reverso, y los reportes excluyen las anuladas del total recaudado.

22.6. Tabla detalle_venta

Campo	Tipo	Clave	Descripcion
id_detalle	INT	PK	Identificador unico del detalle.
id_venta	INT	FK	Venta a la que pertenece.
id_producto	INT	FK	Producto vendido.
cantidad	INT		Unidades vendidas.
precio_unitario	DECIMAL(10,2)		Precio aplicado al momento de la venta.
subtotal	DECIMAL(10,2)		Cantidad por precio unitario.

Nota. El precio se copia en el detalle y no se consulta del producto. Si se consultara, cambiar el precio de un articulo alteraria el importe de todas las ventas pasadas.

22.7. Tabla movimientos_stock

Campo	Tipo	Clave	Descripcion
id_movimiento	INT	PK	Identificador unico del movimiento.
id_producto	INT	FK	Producto afectado.
tipo	ENUM		Entrada, salida, ajuste, venta o baja.
cantidad	INT		Cantidad modificada.

Campo	Tipo	Clave	Descripcion
motivo	VARCHAR(255)		Descripcion del movimiento.
fecha_hora	DATETIME		Momento del cambio de stock.
id_usuario	INT	FK	Usuario que genera o registra el movimiento.

22.8. Tabla bajas_producto

Campo	Tipo	Clave	Descripcion
id_baja	INT	PK	Identificador unico de la baja.
id_producto	INT	FK	Producto vencido o danado.
cantidad	INT		Unidades retiradas del stock.
motivo	ENUM		Vencido, danado u otro motivo justificado.
fecha_hora	DATETIME		Momento de la baja.
id_usuario	INT	FK	Usuario que registra la baja.

23. Reglas de integridad

40. No se debe permitir una venta si el stock disponible es menor a la cantidad solicitada.
41. Cada venta debe tener al menos un registro en detalle_venta.
42. El subtotal de cada detalle se calcula como cantidad por precio unitario.
43. El total de la venta se calcula como la suma de los subtotales.
44. Cada venta confirmada debe descontar stock y registrar un movimiento de tipo venta.
45. Cada baja por vencimiento o dano debe descontar stock y registrar su trazabilidad.
46. El codigo de barras debe ser unico cuando se informe.
47. El email de usuario debe ser unico, para permitir la autenticacion.
48. Una venta anulada no puede volver a anularse, y su anulacion repone el stock de todos sus detalles.

49. Un producto dado de baja logica no puede seleccionarse en una venta nueva, pero sigue figurando en las ventas anteriores.

Las diez reglas se verifican en el backend, es decir, en el servidor, y no unicamente en la interfaz. La razon es que una validacion que vive solo en el navegador puede evitarse enviando la peticion directamente a la interfaz de programacion de aplicaciones, de modo que no constituye una garantia de integridad. Cada regla cuenta, ademas, con al menos una prueba automatizada que verifica su caso de incumplimiento.

24. Diseno de clases

El diseno distingue dos familias de clases. Las clases de dominio representan los conceptos del negocio y contienen las reglas que dependen unicamente de sus propios datos. Las clases de servicio coordinan varias entidades y aplican las reglas que involucran mas de un concepto.

Tabla 14
Clases de dominio

Clase	Responsabilidad	Atributos principales	Metodos principales
Usuario	Representar a propietarios y empleados.	id, nombre, email, passwordHash, rol, activo	validarRol(), estaActivo()
Producto	Representar articulos del inventario.	id, codigoBarra, nombre, precio, stockActual, stockMinimo, fechaVencimiento, activo	actualizarStock(), estaBajoStock(), estaVencido()
Categoria	Agrupar productos.	id, nombre, descripcion, activo	activar(), desactivar()
MedioPago	Representar formas de cobro.	id, nombre, activo	habilitar(), deshabilitar()
Venta	Representar una operacion comercial.	id, fechaHora, usuario, medioPago, detalles, total, estado	agregarDetalle(), calcularTotal(), confirmar(), anular()
DetalleVenta	Representar producto y cantidad dentro de una venta.	producto, cantidad, precioUnitario, subtotal	calcularSubtotal()
MovimientoStock	Registrar la trazabilidad del inventario.	producto, tipo, cantidad, motivo, fechaHora, usuario	registrar()

Clase	Responsabilidad	Atributos principales	Metodos principales
BajaProducto	Registrar productos vencidos o danados.	producto, cantidad, motivo, fechaHora, usuario	validarCantidad(), aplicarBaja()

Tabla 15
Clases de servicio

Clase	Responsabilidad	Metodos principales
AuthService	Gestionar el inicio de sesion y la validacion de la sesion.	login(), validarCredenciales(), validarToken()
ProductoService	Gestionar el alta, la modificacion, la baja y la busqueda de productos.	crear(), actualizar(), darDeBaja(), buscar()
VentaService	Aplicar las reglas de negocio de ventas.	registrarVenta(), validarStock(), anularVenta()
StockService	Registrar bajas y movimientos de inventario.	registrarBaja(), registrarMovimiento(), alertasDeStock()
ReporteService	Generar reportes y estadisticas.	ventasPorPeriodo(), productosMasVendidos()

Nota. El metodo estaVencido() de la clase Producto fue el origen del defecto DEF-002: comparaba la fecha de vencimiento interpretada en horario universal contra la medianoche local, de modo que un producto que vencia el mismo dia se informaba como ya vencido. El detalle figura en la seccion 34.

25. Flujo entre capas

La aplicacion esta organizada en capas y cada peticion las recorre siempre en el mismo orden. Esta separacion es la que da cumplimiento al requerimiento no funcional RNF-06, porque permite modificar una capa sin alterar las demas.

Tabla 16
Capas de la aplicacion y su funcion

Capa	Componentes	Funcion
Frontend	Pantallas de ingreso, tablero, productos, venta, bajas, reportes, mas vendidos e historial.	Permite la interaccion del usuario con las funcionalidades principales.
Rutas	Definicion de las 21 rutas de la interfaz de programacion de aplicaciones.	Asocian cada direccion con su controlador y declaran que rol puede utilizarla.
Middleware	Autenticacion, control de rol, limite de intentos y manejo de errores.	Resuelven lo que es comun a muchas rutas antes de llegar al controlador.

Capa	Componentes	Funcion
Controladores	AuthController, ProductoController, VentaController, StockController y ReporteController.	Reciben las solicitudes, validan los datos basicos y llaman a los servicios.
Servicios	AuthService, ProductoService, VentaService, StockService y ReporteService.	Ejecutan las reglas de negocio y coordinan las operaciones.
Repositorios	Repositorios de usuario, producto, venta, stock y reportes.	Centralizan las consultas y la persistencia, y aislan al resto del sistema del motor de base de datos.
Base de datos	MySQL 8 con las ocho tablas del modelo.	Almacena usuarios, productos, ventas, stock y la informacion consultable por los reportes.

La capa de repositorios cumple una funcion adicional que conviene explicitar. Al concentrar en un unico lugar todo el acceso a los datos, el sistema puede funcionar con dos implementaciones intercambiables detras de la misma interfaz: una contra MySQL y otra en memoria, que carga el mismo juego de datos de prueba. Esa decision permite ejecutar las pruebas automatizadas y la demostracion sin necesidad de un servidor de base de datos, sin modificar una sola linea de los servicios ni de los controladores. El modo se selecciona mediante una variable de entorno.

26. Metodologia de desarrollo

El equipo adopto un enfoque iterativo e incremental. No se siguio un marco de trabajo formal completo, sino un conjunto de practicas tomadas de los metodos agiles y adaptadas al tamano del equipo, a la carga academica de sus integrantes y a la duracion del proyecto.

26.1. Organizacion del trabajo

El trabajo se organizo por tickets. Cada ticket representa un objetivo funcional completo, tiene un responsable unico y da origen a una rama propia en el repositorio. Esta correspondencia

entre ticket, rama y responsable es la que permite reconstruir, a partir del historial, quien hizo cada cosa y cuando.

La asignacion de tickets siguio las especialidades declaradas en la seccion 10, de modo que cada integrante trabajo mayoritariamente sobre archivos distintos. Esa decision redujo los conflictos de integracion: en todo el proyecto no se registro ningun conflicto que requiriera resolucion manual.

26.2. Ciclo de trabajo de cada cambio

Todo cambio incorporado al sistema recorrio el siguiente ciclo:

50. El responsable del ticket crea una rama a partir de la rama principal actualizada.
51. Formula el prompt que describe el problema a resolver, con su contexto y sus restricciones, y lo registra.
52. Revisa lo que produjo el asistente de inteligencia artificial, lo ajusta al estilo y al alcance del proyecto y descarta lo que no corresponde.
53. Ejecuta las pruebas automatizadas de forma local. Si alguna falla, el cambio no avanza.
54. Confirma el cambio con un mensaje que menciona el ticket y describe el efecto, no la implementacion.
55. Publica la rama y abre una solicitud de incorporacion de cambios hacia la rama principal.
56. La integracion continua ejecuta las pruebas, compila el frontend y verifica el esquema de la base de datos. Si algun trabajo falla, la incorporacion queda bloqueada.
57. Una vez en verde, el cambio se integra a la rama principal.

26.3. Practicas adoptadas y su motivo

Tabla 17
Practicas de desarrollo adoptadas por el equipo

Practica	Motivo de su adopcion
Rama por ticket e integracion mediante solicitud de cambios	La rama principal se mantiene siempre en un estado que funciona, y cada integracion queda documentada con su revision.
Validacion de las reglas de negocio en el servidor	Una validacion que vive solo en el navegador puede evitarse enviando la peticion directamente a la interfaz de programacion de aplicaciones.
Comentar el motivo y no la accion	El codigo ya dice lo que hace. Lo que se pierde con el tiempo es por que se hizo de esa manera, y es eso lo que evita que alguien deshaga una correccion pensando que sobra.
Una prueba automatizada por cada caso limite	Los dos defectos de mayor severidad del proyecto no se detectaron leyendo el codigo, sino ejecutando pruebas sobre sus casos de borde.
Registro de defectos con ficha completa	Permite distinguir un defecto corregido de uno que solo dejo de manifestarse, y da evidencia verificable del trabajo de aseguramiento de la calidad.
Integracion continua obligatoria	Evita que un cambio que no compila o que rompe una prueba llegue a la rama principal, con independencia de la buena voluntad de quien lo propone.

27. Planificacion de tareas

Tabla 18
Etapas, tareas, responsables y entregables

Etapa	Tareas principales	Responsable	Entregable
Relevamiento	Validar la entrevista, los requerimientos y el alcance.	Equipo completo	Requerimientos validados.
Analisis	Casos de uso, historias de usuario, modelo de datos y clases.	Aaron y Leonardo	Documentacion tecnica.
Diseno	Wireframes, estructura de la base y arquitectura.	Lucas y Leonardo	Wireframes y diagrama entidad-relacion.
Desarrollo frontend	Pantallas de ingreso, tablero, productos, ventas y reportes.	Lucas	Componentes React.
Desarrollo backend	Rutas, reglas de negocio, autenticacion, ventas y reportes.	Elian	Interfaz de programacion de aplicaciones en Node.js.
Base de datos	Tablas, relaciones, consultas y datos de prueba.	Leonardo	Scripts SQL y consultas.
Testing	Casos de prueba, registro de errores y validacion de requerimientos.	Aaron	Plan y evidencias de prueba.
Integracion continua	Automatizacion de la verificacion en cada cambio.	Aaron	Flujo de integracion continua.

Etapa	Tareas principales	Responsable	Entregable
Entrega	Integracion final, documentacion y presentacion del sistema en funcionamiento.	Equipo completo	Sistema terminado y documento final.

28. Diagrama de Gantt

El cronograma se expresa a continuacion en semanas, tomando como referencia las fechas efectivas que registra el historial del repositorio. La primera semana corresponde al 27 de agosto de 2026, fecha de creacion de la rama principal, y la ultima al 13 de octubre de 2026, fecha de esta entrega.

Tabla 19
Cronograma de ejecucion por semana

Etapa	S1	S2	S3	S4	S5	S6	S7	S8
Relevamiento y analisis	X	X						
Diseno y wireframes		X	X					
Estructura inicial del sistema		X	X					
Plan y casos de prueba			X	X				
Base de datos				X	X			
Pruebas automatizadas				X	X			
Desarrollo frontend					X	X	X	
Integracion continua						X		
Auditoria y correcciones						X	X	X
Documentacion final y entrega							X	X

Nota. El diagrama de Gantt original, elaborado con la herramienta Lucidchart, se adjunta como anexo grafico. La tabla precedente refleja la ejecucion real y no la planificacion inicial.

29. Gestion de riesgos

Tabla 20
Riesgos identificados, mitigacion y resultado

ID	Riesgo	Prob.	Impacto	Mitigacion	Resultado
R-01	Errores de stock por ventas concurrentes o validaciones incompletas.	Media	Alto	Validar el stock en el backend y registrar los movimientos de inventario.	Mitigado. El procedimiento de bajas bloquea la fila del producto y 13 pruebas verifican las reglas de stock.
R-02	Consultas de reportes lentas con muchos registros.	Baja	Medio	Indexar fechas, productos y ventas, y probar las consultas con datos de ejemplo.	Mitigado. Se crearon seis indices de apoyo sobre las columnas que usan los reportes.
R-03	Interfaz poco clara para usuarios no tecnicos.	Media	Medio	Validar los wireframes con el cliente y priorizar pantallas simples.	Mitigado parcialmente. Resta el acta de validacion firmada por el cliente.
R-04	Falta de pruebas antes de la entrega.	Media	Alto	Preparar casos de prueba por requerimiento y ejecutar la lista de verificacion antes de presentar.	Mitigado. 129 casos documentados y 78 pruebas automatizadas que se ejecutan en cada cambio.
R-05	Dependencia excesiva de respuestas de inteligencia artificial sin revision.	Media	Medio	Revisar el codigo generado, ejecutar las pruebas y registrar los prompts utilizados.	Materializado y controlado. Dos de los tres defectos registrados tienen este origen; ambos fueron detectados por las pruebas automatizadas y corregidos.

El riesgo R-05 merece un comentario, porque es el unico que efectivamente se materializo. Su impacto no fue mayor precisamente porque la mitigacion prevista estaba en funcionamiento: los defectos DEF-002 y DEF-003 corresponden a codigo generado con asistencia de inteligencia artificial que resultaba incorrecto en casos de borde, y ambos fueron detectados por pruebas automatizadas antes de la entrega. La leccion registrada es que la revision por lectura no alcanza para esta clase de errores.

30. Plan de testing

30.1. Alcance

El objetivo del plan de pruebas es validar el correcto funcionamiento del sistema Kiosky frente a los catorce requerimientos funcionales y las seis historias de usuario definidas. El responsable principal es Aaron Brumat, en el marco del ticket KIO-08, con la colaboracion del equipo completo para la correccion de los defectos detectados.

Quedan dentro del alcance:

- Los cinco casos de uso con sus flujos principales y alternativos.
- Las diez reglas de integridad enunciadas en la seccion 23.
- El control de acceso por rol: que un empleado no pueda realizar las operaciones reservadas al propietario.
- Los calculos de subtotales, totales y periodos de los reportes.
- Los requerimientos no funcionales verificables: tiempo de respuesta, compatibilidad entre navegadores y proteccion del acceso.

Quedan fuera del alcance, por exceder lo previsto para la materia:

- Pruebas de carga con volumenes de datos de produccion.
- Pruebas de seguridad ofensivas o de intrusion.
- Pruebas de compatibilidad con navegadores sin soporte vigente.

30.2. Ambiente de pruebas

Tabla 21
Ambiente utilizado para la ejecucion de las pruebas

Componente	Configuracion
Frontend	React 18 con Vite, ejecutado en Google Chrome, Microsoft Edge y Mozilla Firefox.
Backend	Node.js 20 con Express.
Base de datos	MySQL 8 con el juego de datos de prueba, y modo en memoria con el mismo juego de datos para las pruebas automatizadas.

Componente	Configuracion
Ejecutor de pruebas	Ejecutor nativo de Node.js, sin dependencias externas.
Integracion continua	GitHub Actions, con un servicio MySQL 8 efimero.

30.3. Estrategia y tipos de prueba

58. **Pruebas funcionales manuales.** Validacion de la interfaz y de las reglas de negocio caso por caso, sobre el sistema en ejecucion.
59. **Pruebas de integracion.** Verificacion de la comunicacion entre los componentes React, las rutas de Node.js y la persistencia en MySQL.
60. **Pruebas de reglas de integridad y de negocio.** Comprobacion del bloqueo de venta sin stock, el calculo de subtotales, la actualizacion automatica del inventario y la trazabilidad de las bajas.
61. **Pruebas automatizadas.** Ejecucion repetible de los casos criticos del plan, descrita en la seccion 32.

30.4. Juego de datos y valores de referencia

Las pruebas se ejecutan sobre un juego de datos fijo, compuesto por dos usuarios, cinco categorias, doce productos, cuatro medios de pago y siete ventas precargadas, una de ellas anulada. Las fechas del juego de datos son relativas al dia de ejecucion, de modo que los valores esperados no caducan con el paso del tiempo.

Contar con valores de referencia exactos es lo que permite distinguir un reporte correcto de uno que simplemente devuelve algun numero. Los siguientes son los valores que debe arrojar el sistema sobre el juego de datos inicial:

Tabla 22
Valores esperados sobre el juego de datos inicial

Indicador	Dia	Semana	Mes
Total recaudado	$14.500	$29.500	$36.900
Cantidad de ventas	3	5	6
Ticket promedio	$4.833,33	$5.900	$6.150

Nota. La venta anulada no se cuenta en ninguno de los tres periodos. Los totales que la incluyeran indicarian un incumplimiento de la novena regla de integridad.

Tabla 23
Alertas y ranking esperados sobre el juego de datos inicial

Indicador	Valor esperado
Productos con stock menor o igual al minimo	4
Productos proximos a vencer, dentro de 30 dias	3
Productos ya vencidos	1
Producto mas vendido	Chocolate Barra 30g, con 11 unidades

30.5. Criterio de aceptacion de la entrega

- El 100 % de los casos de severidad alta finaliza en estado aprobado.
- No quedan defectos abiertos de severidad alta o critica.
- Las pruebas automatizadas se ejecutan sin fallos en la rama principal.
- La integracion continua se encuentra en verde en el ultimo cambio integrado.

31. Casos de prueba

El plan de pruebas se documento en tres piezas que se complementan y que se encuentran en el repositorio, en la carpeta de documentacion de testing. En total suman 129 casos de prueba documentados.

Tabla 24
Documentos del plan de pruebas

Documento	Casos	Contenido
Plan de pruebas funcionales	48	Casos derivados de los cinco casos de uso, organizados por flujo principal y flujo alternativo, con criterios de aceptacion.
Casos de prueba manuales por requerimiento	81	Casos organizados por requerimiento funcional, con datos de entrada, resultado esperado y matriz de trazabilidad hacia los RF.
Documentacion tecnica de testing	-	Alcance, ambiente, juego de datos, valores de referencia, procedimiento de ejecucion y registro de defectos.

31.1. Casos representativos

A modo de sintesis se presentan diez casos representativos, seleccionados por cubrir un flujo principal, un flujo alternativo o una regla de integridad de cada caso de uso. La nomenclatura completa de los 129 casos figura en los documentos citados.

Tabla 25
Casos de prueba representativos y su resultado

ID	RF o CU	Objetivo	Resultado esperado	Estado
TC-01	RF-01	Inicio de sesion exitoso.	Acceso concedido, redireccion al tablero y generacion de la sesion.	Aprobado
TC-02	RF-01	Inicio de sesion fallido.	Mensaje de credenciales invalidas. No permite el ingreso.	Aprobado
TC-03	CU-02	Alta de producto correcta.	Producto guardado y visible en la tabla de inventario.	Aprobado
TC-04	CU-02	Alta con codigo de barras duplicado.	Error de restriccion de unicidad, con mensaje que indica que el codigo ya pertenece a otro articulo.	Aprobado
TC-05	CU-01	Registro de venta normal.	Registro en ventas y en detalle_venta, descuento automatico del stock y creacion del movimiento de inventario.	Aprobado
TC-06	CU-01 FA-1	Venta con stock insuficiente.	El sistema bloquea la venta e informa el stock disponible.	Aprobado
TC-07	CU-03	Registro de producto vencido o danado.	Insercion en bajas_producto, descuento del stock y registro del movimiento de tipo baja.	Aprobado

ID	RF o CU	Objetivo	Resultado esperado	Estado
TC-08	CU-04	Consulta de reporte de ventas.	Devuelve el total vendido, el desglose por medio de pago y el detalle de operaciones del periodo.	Aprobado
TC-09	CU-05	Ranking de productos mas vendidos.	Lista ordenada de forma descendente por unidades vendidas acumuladas.	Aprobado
TC-10	RF-13	Alerta de stock bajo en el tablero.	La pantalla principal resalta los productos cuyo stock es menor o igual al minimo.	Aprobado

32. Pruebas automatizadas

Los casos criticos del plan fueron automatizados para poder ejecutarlos en cualquier momento y en cada cambio. La suite consta de 65 pruebas y se ejecuta, en una computadora de escritorio, en menos de un segundo.

Se utilizo el ejecutor de pruebas nativo de Node.js, sin incorporar ninguna dependencia externa. La decision tiene un motivo concreto: evitar que la ejecucion de las pruebas dependa de la instalacion de paquetes adicionales, de modo que cualquier integrante pueda correrlas con el mismo comando y sin configuracion previa.

Tabla 26
Distribucion de las pruebas automatizadas

Conjunto	Pruebas	Que verifica
Dominio	24	Reglas que dependen de una sola entidad: calculo de subtotales y totales, alerta de stock bajo, deteccion de vencimiento y validacion de rol.
Ventas	13	Registro de venta, bloqueo por stock insuficiente, descuento de inventario, anulacion y reposicion de stock.
Stock	13	Bajas por vencimiento y dano, validacion de cantidad, trazabilidad de movimientos y alertas.
Reportes	15	Totales por dia, semana y mes, ticket promedio, exclusion de ventas anuladas y ranking de productos.

Nota. La distribucion no es casual: los conjuntos de ventas y de stock concentran los casos negativos, porque son los que verifican las reglas de integridad que el sistema debe hacer cumplir.

Conviene senalar el valor concreto que aportaron estas pruebas, mas alla de su cantidad. Dos de los tres defectos registrados en el proyecto fueron descubiertos por ellas y no por la revision manual: DEF-002 aparecio al escribir la prueba del producto que vence el mismo dia, y DEF-003 al verificar los reportes contra una base de datos con el reloj desplazado. Ninguno de los dos se manifestaba en el uso normal del sistema, y ambos habrian llegado a la entrega sin este trabajo.

33. Integracion continua

El repositorio cuenta con un flujo de integracion continua que se ejecuta automaticamente en cada cambio enviado y en cada solicitud de incorporacion hacia la rama principal. Esta compuesto por tres trabajos independientes.

Tabla 27
Trabajos del flujo de integracion continua

Trabajo	Que ejecuta y que garantiza
Pruebas	Instala las dependencias del backend y ejecuta la suite completa de pruebas automatizadas. Garantiza que ningun cambio rompa una regla de negocio ya verificada.
Frontend	Compila el frontend para produccion. Garantiza que el cambio no introduzca un error que impida construir la aplicacion, algo que el servidor de desarrollo puede tolerar sin avisar.
Base de datos	Levanta un servicio MySQL 8 desde cero, aplica el script del esquema y el juego de datos de prueba, inicia la interfaz de programacion de aplicaciones contra esa base real y verifica que responda e informe el modo MySQL, y que el catalogo devuelva los doce productos esperados.

El tercer trabajo es el que mas aporta y merece una explicacion. Las pruebas automatizadas se ejecutan en modo memoria, lo que las hace rapidas pero deja un hueco: no verifican que el script del esquema documentado se aplique efectivamente sobre una base vacia, ni que las consultas SQL sean validas para MySQL. Ese trabajo cierra el hueco levantando una base real en cada ejecucion. Un error de sintaxis en el esquema, o una columna que el backend consulta pero que el script no crea, quedan detenidos ahi y no llegan a la rama principal.

34. Registro de defectos y auditoria interna

Durante la ejecucion del plan de pruebas y la auditoria interna del codigo se registraron tres defectos. Se informan los tres, incluido el de severidad baja, porque el registro completo es parte de la evidencia del trabajo de aseguramiento de la calidad.

34.1. Clasificacion utilizada

Tabla 28
Criterio de severidad aplicado

Severidad	Definicion
Critica	Impide el uso del sistema o provoca perdida de datos.
Alta	Una funcionalidad central devuelve un resultado incorrecto.
Media	Una funcionalidad secundaria devuelve un resultado incorrecto, o una central falla en un caso de borde.
Baja	El comportamiento es correcto pero fragil, poco verificable o dependiente de una conducta implicita.

34.2. Defectos registrados

Tabla 29
Registro de defectos del proyecto

ID	Fecha	Severidad	Descripcion	Estado
DEF-001	01/09/2026	Baja	La confirmacion por codigo de barras dependia unicamente del envio implicito del formulario al presionar Enter. Funciona en un navegador real, pero deja el comportamiento sujeto a una conducta implicita y dificulta verificarlo de forma automatizada.	Corregido
DEF-002	10/09/2026	Media	Un producto cuya fecha de vencimiento es el dia de hoy se informaba como ya vencido. Error de un dia en toda la logica de vencimientos, por comparar una fecha interpretada en horario universal contra la medianoche local.	Corregido
DEF-003	20/09/2026	Alta	Las ventas se guardaban con el reloj del servidor de base de datos mientras los periodos de los reportes se calculaban con el reloj de la aplicacion. Con los relojes en husos distintos, el reporte del dia no incluia las ventas recientes.	Corregido

Detalle de DEF-003

Por su severidad, este defecto se documenta con su verificacion completa. El sintoma: una venta registrada a las 21:30 no aparecia en el reporte del dia. La causa: la sentencia que graba la venta toma la hora del servidor de base de datos, mientras que el periodo del reporte se calcula con la fecha local del proceso de la aplicacion. Si el contenedor de la base corre en horario universal y la aplicacion en horario de Argentina, la venta queda grabada con la fecha del dia siguiente y el reporte del dia la excluye.

La verificacion se realizo forzando el desfasaje de forma deliberada: se configuro la base de datos en el huso +09:00 mientras la aplicacion corria en el huso local. Con el reloj de la base en el 13 de octubre a las 06:21 y el de la aplicacion en el 12 de octubre a las 18:21, una venta registrada a traves de la interfaz de programacion de aplicaciones quedo almacenada con fecha 12 de octubre a las 18:21, y el reporte del dia devolvio correctamente una venta por $1.200. Antes de la correccion, el mismo escenario devolvia cero ventas.

34.3. Hallazgos de la auditoria interna

Ademas de los defectos funcionales, la auditoria del codigo detecto cuatro situaciones que no producian un resultado incorrecto pero que convenia resolver antes de la entrega.

Tabla 30
Hallazgos de la auditoria interna y su resolucion

Hallazgo	Descripcion	Resolucion
Intentos de ingreso sin limite	Era posible probar contrasenas de forma indefinida contra la pantalla de ingreso, sin que el sistema opusiera ninguna demora.	Limite de cinco intentos fallidos por cuenta cada quince minutos, aplicado en la ruta de ingreso.
Columna del modelo sin documentar	La columna que implementa la baja logica de productos se habia incorporado al esquema durante la implementacion, pero no figuraba en el	Documentada en el diccionario de datos de la seccion 22, con la justificacion de la decision.

Hallazgo	Descripcion	Resolucion
	diccionario de datos, de modo que el documento y la base no coincidian.	

El segundo hallazgo merece una aclaracion, porque su origen fue un error de documentacion y no de programacion. El trabajo de base de datos seguia fielmente el diccionario de datos tal como estaba definido en su momento. La columna de baja logica fue agregada despues, durante la implementacion, sin incorporarla al diccionario. Es decir: la discrepancia no provino del diseno de la base, sino de una modificacion del modelo que no se documento cuando se hizo. Esta entrega corrige esa omision, y deja ademas asentado el motivo de la decision, que es lo que evita que vuelva a perderse.

35. Entorno operativo

Se distinguen tres entornos con proposito y configuracion diferentes.

Tabla 31
Entornos del sistema

Entorno	Proposito	Configuracion
Desarrollo	Trabajo diario de cada integrante.	Computadora personal con Node.js 20, MySQL 8 local o modo memoria, y el servidor de desarrollo de Vite. No requiere configuracion previa: el sistema arranca con valores por defecto.
Integracion continua	Verificacion automatica de cada cambio.	Maquina virtual efimera provista por GitHub Actions, con Node.js 20 y un servicio MySQL 8 que se crea y se destruye en cada ejecucion.
Demostracion	Revision del sistema en funcionamiento por parte del docente y del cliente.	El proyecto se abre en Visual Studio Code y se ejecutan el backend y el frontend en la misma computadora. El procedimiento se detalla en la seccion 36.

35.1. Requisitos de hardware y software

Para ejecutar el sistema:

- Computadora personal con Windows, macOS o Linux.

- Node.js 20 o superior, que provee tambien el gestor de paquetes y el ejecutor de pruebas.
- Visual Studio Code, o cualquier editor equivalente.
- MySQL 8, unicamente cuando se desea trabajar con persistencia en base de datos. El sistema tambien funciona sin el.
- Memoria disponible: el backend en modo memoria utiliza aproximadamente 22 MB.

Para utilizarlo:

- Navegador con soporte vigente: Google Chrome, Microsoft Edge o Mozilla Firefox.
- Resolucion minima de 360 pixeles de ancho. La interfaz es adaptable y se verifico en telefono, tableta y escritorio.

Conviene senalar que el sistema no exige configuracion previa para arrancar. Los valores por defecto alcanzan, y si no encuentra una base de datos disponible continua con el juego de datos de prueba cargado en memoria, dejando constancia del motivo por consola. Esa decision fue deliberada: cualquier integrante, o quien evalue el proyecto, puede clonar el repositorio y tener el sistema funcionando con dos comandos.

36. Implementacion y despliegue

36.1. Tecnologias utilizadas

El sistema web Kiosky se desarrollo con React 18 y Vite para el frontend, complementado con HTML5, CSS3 y JavaScript. El backend se implemento en Node.js con Express, y la gestion de la base de datos se realiza con MySQL 8. La autenticacion utiliza tokens firmados y las contrasenas se almacenan con una funcion de resumen con sal, nunca en texto plano.

Para la organizacion del codigo se definio la arquitectura en capas descrita en la seccion 25. Para el control de versiones, la revision de codigo y el trabajo colaborativo mediante ramas independientes se utiliza Git junto con GitHub, con integracion continua. Para la gestion general del proyecto y el seguimiento de los tickets se emplea Trello. El desarrollo se apoya en Visual Studio Code como entorno principal, con asistentes de inteligencia artificial integrados.

36.2. Estructura del sistema entregado

Tabla 32
Componentes del sistema entregado

Componente	Detalle
Pantallas del frontend	Ocho: ingreso, tablero, productos, nueva venta, bajas, reportes, mas vendidos e historial.
Rutas de la interfaz de programacion	21, agrupadas en autenticacion, productos, ventas, stock y reportes.
Tablas de la base de datos	Ocho, con seis indices de apoyo, un procedimiento almacenado y dos vistas analiticas.
Clases de dominio y de servicio	Ocho de dominio y cinco de servicio.
Pruebas automatizadas	65.
Casos de prueba documentados	129.

36.3. Puesta en marcha del sistema

El sistema se ejecuta localmente, abriendo el proyecto en Visual Studio Code. Son dos procesos que corren a la vez: el backend, que expone la interfaz de programacion de aplicaciones, y el frontend, que el servidor de desarrollo de Vite publica en el navegador. El procedimiento completo es el siguiente:

62. Clonar el repositorio y abrir la carpeta en Visual Studio Code.
63. En una terminal, entrar en la carpeta del backend, instalar las dependencias y arrancarlo. Queda escuchando en el puerto 4000.

64. En una segunda terminal, entrar en la carpeta del frontend, instalar las dependencias y arrancar el servidor de desarrollo. Queda disponible en el puerto 5173.
65. Abrir esa direccion en el navegador e ingresar con alguno de los dos usuarios del juego de datos de prueba.

El servidor de desarrollo del frontend redirige al backend todas las peticiones cuya direccion comienza con la ruta de la interfaz, de modo que para el navegador ambos conviven en el mismo origen y no hay que configurar nada mas.

La demostracion se ejecuta en modo memoria, decision que responde a dos motivos:

66. No requiere tener un servidor MySQL instalado y en funcionamiento, de modo que el sistema arranca en cualquier computadora con Node.js y en pocos segundos.
67. El sistema vuelve al juego de datos inicial en cada reinicio, de modo que la demostracion siempre comienza en un estado conocido y los valores de referencia de la seccion 30 son verificables en cualquier momento.

El soporte de MySQL permanece intacto y se selecciona con una variable de entorno; el trabajo de base de datos de la integracion continua lo verifica en cada cambio contra un servidor real. La interfaz informa en pantalla cuando el sistema se encuentra en modo de demostracion, de manera que la condicion nunca queda oculta para quien lo revisa.

36.4. Que se puede comprobar en la demostracion

Como el juego de datos es siempre el mismo, la demostracion no se limita a mostrar pantallas: permite verificar resultados contra los valores documentados de antemano en la seccion 30. Un reporte que devuelve algun numero no prueba nada; uno que devuelve exactamente el numero esperado, si.

68. Registrar una venta de dos productos y comprobar que el total se calcula solo y que el stock de ambos disminuye en el catalogo.
69. Intentar vender mas unidades de las disponibles y comprobar que el sistema rechaza la operacion e informa el stock real.
70. Consultar el reporte del dia y contrastarlo con los valores de la tabla 22.
71. Ingresar como empleado e intentar acceder a los reportes, para comprobar que el control por rol se aplica.

37. Evidencia de participacion de los integrantes

Esta seccion da cumplimiento al requisito de que todos los integrantes presenten evidencia de su participacion. La evidencia es verificable de forma independiente: el historial del repositorio registra autor, fecha y contenido de cada cambio, y cada solicitud de incorporacion conserva su revision y el resultado de la integracion continua.

Tabla 33
Participacion por integrante

Integrante	Tickets	Aporte principal	Evidencia
Aaron Brumat	KIO-08	Plan de pruebas, 129 casos documentados, 65 pruebas automatizadas, integracion continua, registro y cierre de defectos, y la documentacion del proyecto.	11 commits propios en la rama principal
Lucas Beim	KIO-06, KIO-07, KIO-01	Pantalla del tablero con ventas del dia y alertas de stock, y el limite de intentos de ingreso.	6 commits propios en la rama principal
Leonardo Vertali	KIO-03, KIO-04, KIO-05	Esquema de la base de datos, procedimiento almacenado de bajas, vistas analiticas e indices de apoyo.	4 commits propios en la rama principal
Elian Cavenatti	KIO-01, KIO-02	Integracion entre la interfaz y la base de datos, autenticacion y flujo de registro de ventas.	Secciones 13, 14 y 25 de este documento

Nota. El recuento corresponde a los commits propios de cada integrante, sin contar los de integracion de ramas. Cada commit conserva en el historial su autor, su fecha, su mensaje y el detalle de los archivos modificados, y cada solicitud de incorporacion de cambios conserva su revision y el resultado de la integracion continua.

Cabe consignar, por transparencia, que el commit de base del repositorio, identificado como 9c5ee99, incorpora la estructura inicial completa del sistema y fue realizado por Aaron Brumat durante la integracion inicial del proyecto. Los aportes posteriores de cada integrante se construyeron sobre esa base, y su detalle figura en el registro de prompts de la seccion 20.

38. Conclusiones

El sistema Kiosky cumple los catorce requerimientos funcionales y los seis requerimientos no funcionales definidos durante el relevamiento. Cada requerimiento cuenta con casos de prueba asociados, y los casos criticos se ejecutan de forma automatica en cada cambio enviado al repositorio.

Desde el punto de vista del cliente, las ocho necesidades expresadas en la entrevista encontraron respuesta en el sistema entregado: el registro de ventas reemplaza al cuaderno, el stock se actualiza solo, las alertas anticipan los faltantes y los vencimientos, y los reportes permiten conocer el rendimiento del negocio por periodo y por producto.

Desde el punto de vista academico, el proyecto permitio recorrer el ciclo de vida completo de un desarrollo de software: relevamiento, analisis, diseno, implementacion, pruebas, correccion de defectos, documentacion y puesta en funcionamiento. La leccion mas concreta que deja el trabajo no es tecnica sino metodologica, y surge del riesgo R-05: dos de los tres defectos registrados correspondieron a codigo generado con asistencia de inteligencia artificial que resultaba correcto a la lectura e incorrecto en los casos de borde. Ninguno fue hallado revisando el codigo; ambos aparecieron al escribir una prueba que ejercitaba el limite. La inteligencia

artificial acelero el desarrollo, pero fue el trabajo de verificacion el que evito que esos errores llegaran a la entrega.

El criterio de aceptacion fijado en la seccion 30 se cumple: la totalidad de los casos de severidad alta finaliza en estado aprobado, no quedan defectos abiertos de severidad alta ni critica, las pruebas automatizadas se ejecutan sin fallos sobre la rama principal y la integracion continua se encuentra en verde en el ultimo cambio integrado.

39. Glosario

API o interfaz de programacion de aplicaciones. Conjunto de direcciones que un programa expone para que otros programas soliciten datos u operaciones. En Kiosky es el punto por el que el frontend se comunica con el backend.

Backend. Parte del sistema que se ejecuta en el servidor. Contiene las reglas de negocio y el acceso a la base de datos.

Baja logica. Marcar un registro como inactivo en lugar de eliminarlo, de modo que deje de usarse sin perder la informacion historica que lo referencia.

Clave foranea. Campo que referencia la clave primaria de otra tabla y obliga a que el valor referenciado exista.

Commit. Cambio confirmado en el repositorio, con autor, fecha, mensaje y contenido. Es la unidad del historial.

Caso de borde. Situacion limite de una funcionalidad, como la fecha que vence justo hoy o la cantidad que iguala exactamente el stock disponible. Es donde se concentran los errores que la lectura del codigo no revela.

Framework o marco de trabajo. Estructura que provee herramientas, bibliotecas y convenciones para acelerar el desarrollo.

Frontend. Parte del sistema que se ejecuta en el navegador del usuario y con la que este interactua.

Integracion continua. Practica por la cual cada cambio enviado al repositorio dispara automaticamente la ejecucion de las pruebas y la construccion del sistema.

Middleware. Componente que se ejecuta antes del controlador y resuelve lo que es comun a muchas rutas, como la autenticacion o el control de rol.

Node.js. Entorno de ejecucion gratuito, de codigo abierto y multiplataforma que permite ejecutar JavaScript fuera del navegador.

Modo memoria. Forma de ejecucion en la que el sistema carga el juego de datos de prueba en la memoria del proceso en lugar de leerlo de una base de datos. Permite arrancar sin servidor de base de datos y volver siempre al mismo estado conocido.

Pull request o solicitud de incorporacion de cambios. Propuesta de integrar los cambios de una rama en otra, que habilita su revision antes de aceptarla.

Rama. Linea de trabajo paralela dentro del repositorio, que permite desarrollar sin afectar la version principal.

React. Biblioteca de JavaScript de codigo abierto disenada para construir interfaces de usuario, orientada a facilitar el desarrollo de aplicaciones de una sola pagina.

Repositorio. Almacen del codigo fuente junto con todo su historial de cambios.

Token de sesion. Credencial firmada que el servidor entrega al iniciar sesion y que el navegador presenta en cada peticion posterior para acreditar su identidad.

Vista. Consulta almacenada en la base de datos que se utiliza como si fuera una tabla, para no repetir la misma consulta compleja en varios lugares.

Wireframe. Guia visual que representa la estructura o el esqueleto de una pantalla, sin su diseno grafico definitivo.

40. Referencias

Figma. (s. f.). *Figma*. https://www.figma.com/

Git. (s. f.). *Documentacion de Git*. https://git-scm.com/doc

GitHub. (s. f.). *GitHub Actions documentation*. https://docs.github.com/actions

GitHub. (s. f.). *GitHub Copilot documentation*. https://docs.github.com/copilot

Google. (s. f.). *Gemini para estudiantes*. https://gemini.google/students/

Lucid Software. (s. f.). *Lucidchart*. https://lucid.co/lucidchart

Meta Open Source. (s. f.). *React*. https://react.dev/

Microsoft. (s. f.). *Visual Studio Code documentation*. https://code.visualstudio.com/docs

OpenJS Foundation. (s. f.). *Express*. https://expressjs.com/

OpenJS Foundation. (s. f.). *Node.js documentation*. https://nodejs.org/docs/latest-v20.x/api/

Oracle. (s. f.). *MySQL 8.0 reference manual*. https://dev.mysql.com/doc/refman/8.0/en/

Trello. (s. f.). *Trello*. https://trello.com/

Vite. (s. f.). *Vite*. https://vite.dev/

40.1. Recursos de formacion consultados

Se consigna, por transparencia, el material audiovisual de formacion utilizado por el equipo durante el cursado. No se incluye en la lista de referencias por no contar con datos de autoria verificables.

- Curso de JavaScript. https://youtu.be/1glVfFxj8a4
- Curso de Git y GitHub. https://youtu.be/3GymExBkKjE

- Curso de Node.js. https://youtu.be/yB4n_K7dZV8
- Curso de React. https://youtu.be/7iobxzd_2wY